

SystemVerilog Assignment 2 Extra: File-Based Verification & File Handling

Ziad Mostafa Abdelaziz

July 16, 2026

1 Introduction

This report documents the file-based verification of the `multi_op_processor` combinational DUT. File-based testing is a key methodology in verification environments: a golden reference model is constructed in a high-level modeling utility (Python, C++, or Matlab), outputs are dumped alongside input stimuli to text/binary databases, and these databases are parsed by the SystemVerilog testbench at runtime to drive the DUT and verify behavior against expectations. This completely decouples golden model implementation from SystemVerilog.

2 Technical Choices of Data Types & Sizing

The verification environment is designed around specific SystemVerilog constructs:

- **Packed Struct (`stim_t`):** We chose a packed struct to represent the 10-bit input tokens formatted as `{data_in[7:0], op_sel[1:0]}`. This represents the exact bit-level parsing structure cleanly:

```
typedef struct packed {  
    bit [7:0] data_in;  
    bit [1:0] op_sel;  
} stim_t;
```

- **Dynamic Array (`stim_arr[]`):** Allocated dynamically using the `new[]` operator based on the number of stimulus entries read from the inputs database (200 records). Since files can have arbitrary numbers of entries, a dynamic array is ideal to avoid statically sized limits.
- **Queue (`actual_q[$]`):** Used to chronologically collect and buffer individual evaluated output responses from the DUT. Pushing outputs using `push_back()` allows sequence representation.
- **Associative Array (`expected_aa[int]`):** Used to hold expected outputs read from `expected_Out.txt`, mapped directly by an integer index (representing test case sequence identifier). This provides simple, direct retrieval checks during comparison.

3 Routines and Code Snippets

3.1 File Reading and BOM Handling

Before presenting the procedural Verilog routines, we reference the standard SystemVerilog file opening and reading rules per **IEEE Std 1800-2017 Section 21.3 (File input/output tasks and functions)** [1], which standardizes operations for `$fopen`, `$fscanf`, and `$fclose`. A detailed online reference for these file I/O operations can be found on ChipVerify’s tutorial [2]. Additionally, a query search link to these standards is available at Google Search [3].

The task `read_bin_file` opens the target file, reads input tokens line-by-line as strings, strips the Unicode UTF-8 BOM representation if present at the start of files (checks if string length exceeds expected width and takes a substring), validates correct bit-width, and parses the binary character string to integer:

```
1  task automatic read_bin_file(input string file_name, input int width, ref int q[$]);
2      int fd;
3      string line;
4      int val;
5
6      q.delete();
7      fd = $fopen(file_name, "r");
8      if (fd == 0) begin
9          $fatal(1, "[FATAL] Failed to open file: %s", file_name);
10     end
11
12     while ($fscanf(fd, "%s\n", line) == 1) begin
13         // Strip BOM or header characters if line length is larger than expected width
14         if (line.len() > width) begin
15             line = line.substr(line.len() - width, line.len() - 1);
16         end
17
18         if (line.len() != width) begin
19             $fatal(1, "[FATAL] Length mismatch in file '%s'. Expected width: %0d, Got: %0d",
20                 file_name, width, line.len(), line);
21         end
22
23         val = 0;
24         for (int i = 0; i < line.len(); i++) begin
25             if (line[i] == "1") val = (val << 1) | 1;
26             else if (line[i] == "0") val = (val << 1);
27             else $fatal(1, "[FATAL] Invalid binary character in file '%s' at index %0d", file_name, i);
28         end
29         q.push_back(val);
30     end
31     $fclose(fd);
32 endtask
```

3.2 Loading and Unpacking database

The task `load_file_stimulus` uses `read_bin_file` to read both files, checks for matching record counts, allocates `stim_arr`, unpacks fields to appropriate struct variables, and populates the expected associative array:

```

1  task load_file_stimulus();
2      int inputs_raw[$];
3      int expected_raw[$];
4
5      read_bin_file("doc/inputs.txt", 10, inputs_raw);
6      read_bin_file("doc/expected_Out.txt", 8, expected_raw);
7
8      if (inputs_raw.size() != expected_raw.size()) begin
9          $fatal(1, "[FATAL] Size mismatch between inputs and expected records.");
10     end
11
12     stim_arr = new[inputs_raw.size()];
13
14     for (int i = 0; i < inputs_raw.size(); i++) begin
15         stim_arr[i].data_in = inputs_raw[i][9:2];
16         stim_arr[i].op_sel  = inputs_raw[i][1:0];
17         expected_aa[i]      = expected_raw[i];
18     end
19 endtask

```

4 Full Code Listings

4.1 Design: multi_op_processor.sv

```

1 module multi_op_processor (
2     input  [7:0] data_in,
3     input  [1:0] op_sel,
4     output reg [7:0] data_out
5 );
6
7     always @(*) begin
8         case (op_sel)
9             2'b00: data_out = data_in + 1;           // Increment
10            2'b01: data_out = data_in - 1;           // Decrement
11            2'b10: data_out = ~data_in;               // Bitwise NOT
12            2'b11: data_out = data_in << 1;          // Left shift by 1
13            default: data_out = data_in;
14        endcase
15    end
16
17 endmodule

```

4.2 Testbench: tb_multi_op_processor.sv

```

1 `timescale 1ns/1ns
2
3 module multi_op_processor_tb;
4
5     reg  [7:0] data_in;
6     reg  [1:0] op_sel;

```

```

7  wire [7:0] data_out;
8
9  multi_op_processor dut (
10     .data_in(data_in),
11     .op_sel(op_sel),
12     .data_out(data_out)
13 );
14
15 typedef struct packed {
16     bit [7:0] data_in;
17     bit [1:0] op_sel;
18 } stim_t;
19
20 stim_t stim_arr[];
21 bit [7:0] actual_q[$];
22 bit [7:0] expected_aa[int];
23
24 int num_checks = 0;
25 int num_errors = 0;
26
27 task automatic read_bin_file(input string file_name, input int width, ref int q[$]);
28     int fd;
29     string line;
30     int val;
31
32     q.delete();
33     fd = $fopen(file_name, "r");
34     if (fd == 0) $fatal(1, "[FATAL] Failed to open file: %s", file_name);
35
36     while ($fscanf(fd, "%s\n", line) == 1) begin
37         if (line.len() > width) begin
38             line = line.substr(line.len() - width, line.len() - 1);
39         end
40         if (line.len() != width) begin
41             $fatal(1, "[FATAL] Length mismatch in token.");
42         end
43         val = 0;
44         for (int i = 0; i < line.len(); i++) begin
45             if (line[i] == "1") val = (val << 1) | 1;
46             else if (line[i] == "0") val = (val << 1);
47             else $fatal(1, "Invalid binary char.");
48         end
49         q.push_back(val);
50     end
51     $fclose(fd);
52     $display("[INFO] Successfully read %0d items from file: %s", q.size(), file_name);
53 endtask
54
55 task load_file_stimulus();
56     int inputs_raw[$];
57     int expected_raw[$];
58
59     read_bin_file("doc/inputs.txt", 10, inputs_raw);
60     read_bin_file("doc/expected_Out.txt", 8, expected_raw);

```

```

61
62     if (inputs_raw.size() != expected_raw.size()) begin
63         $fatal(1, "Size mismatch.");
64     end
65
66     stim_arr = new[inputs_raw.size()];
67     for (int i = 0; i < inputs_raw.size(); i++) begin
68         stim_arr[i].data_in = inputs_raw[i][9:2];
69         stim_arr[i].op_sel  = inputs_raw[i][1:0];
70         expected_aa[i]      = expected_raw[i];
71     end
72     $display("[INFO] Loaded %0d stimuli records.", stim_arr.size());
73 endtask
74
75 task drive_stim();
76     for (int i = 0; i < stim_arr.size(); i++) begin
77         data_in = stim_arr[i].data_in;
78         op_sel  = stim_arr[i].op_sel;
79         #1;
80         collect_output_data(data_out);
81     end
82     $display("[INFO] Finished driving all %0d stimulus vectors.", stim_arr.size());
83 endtask
84
85 task collect_output_data(input bit [7:0] out_val);
86     actual_q.push_back(out_val);
87 endtask
88
89 task automatic check_results(ref bit [7:0] exp_aa[int], ref bit [7:0] act_q[$]);
90     num_checks = 0;
91     num_errors = 0;
92
93     for (int i = 0; i < act_q.size(); i++) begin
94         num_checks++;
95         if (exp_aa.exists(i)) begin
96             if (act_q[i] == exp_aa[i]) begin
97                 $display("[CHECK][PASS] idx=%0d inputs={data_in:0x%2h, op_sel:2'b%0b} ",
98                     i, stim_arr[i].data_in, stim_arr[i].op_sel, exp_aa[i], act_q[i]);
99             end else begin
100                 num_errors++;
101                 $display("[CHECK][FAIL] idx=%0d inputs={data_in:0x%2h, op_sel:2'b%0b} ",
102                     i, stim_arr[i].data_in, stim_arr[i].op_sel, exp_aa[i], act_q[i]);
103             end
104         end
105     end
106     $display("\n=== SUMMARY: total_checks=%0d total_errors=%0d ===", num_checks, num_errors);
107     if (num_errors == 0) $display("RESULT: ALL CHECKS PASSED\n");
108     else $display("RESULT: TEST FAILED\n");
109 endtask
110
111 initial begin
112     $display("=== Starting Assignment 2 Extra File-Based Verification ===");
113     load_file_stimulus();
114     drive_stim();

```

```

115         check_results(expected_aa, actual_q);
116         $finish;
117     end
118
119 endmodule

```

4.3 Automation Script: run.do

```

1 vlib work
2 vlog -sv -work work src/multi_op_processor.sv src/tb_multi_op_processor.sv
3 vsim -c work.multi_op_processor_tb
4 add wave *
5 run -all

```

5 Verification & Waveform Results

5.1 Simulation Transcript Log Screenshot

Figure 1 displays the final verification summary log printed in the ModelSim console, proving that all 200 comparison checks passed successfully with zero error mismatches:

```

# === SUMMARY: total_checks=200 total_errors=0 ===
# RESULT: ALL CHECKS PASSED
#
# ** Note: $finish      : E:/courses/Digital IC design Course/Digital Verification Course/system verilog assignments/assignment 2 extra/src/tb_multi_op_processor.sv(160)
# Time: 200 ns Iteration: 0 Instance: /multi_op_processor_tb

```

Figure 1: ModelSim console transcript display showing the successful check results

References

- [1] IEEE Standards Association, *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, Section 21.3. <https://ieeexplore.ieee.org/document/8299595>
- [2] ChipVerify, *SystemVerilog File Input/Output (I/O) Operations*, Available online: <https://www.chipverify.com/systemverilog/systemverilog-file-io>
- [3] Google Search, *Query for SystemVerilog File I/O operations IEEE Std 1800-2017 Section 21.3*, Available online: <https://www.google.com/search?q=SystemVerilog+file+I%2F0+IEEE+Std+1800-2017+Section+21.3>